HTR Platform — Complete Technical & Operations Guide

Audience: Platform owner, content authors, developers, and future team members.
Status: Living document — update when architecture, tooling, or workflows change.

Table of Contents

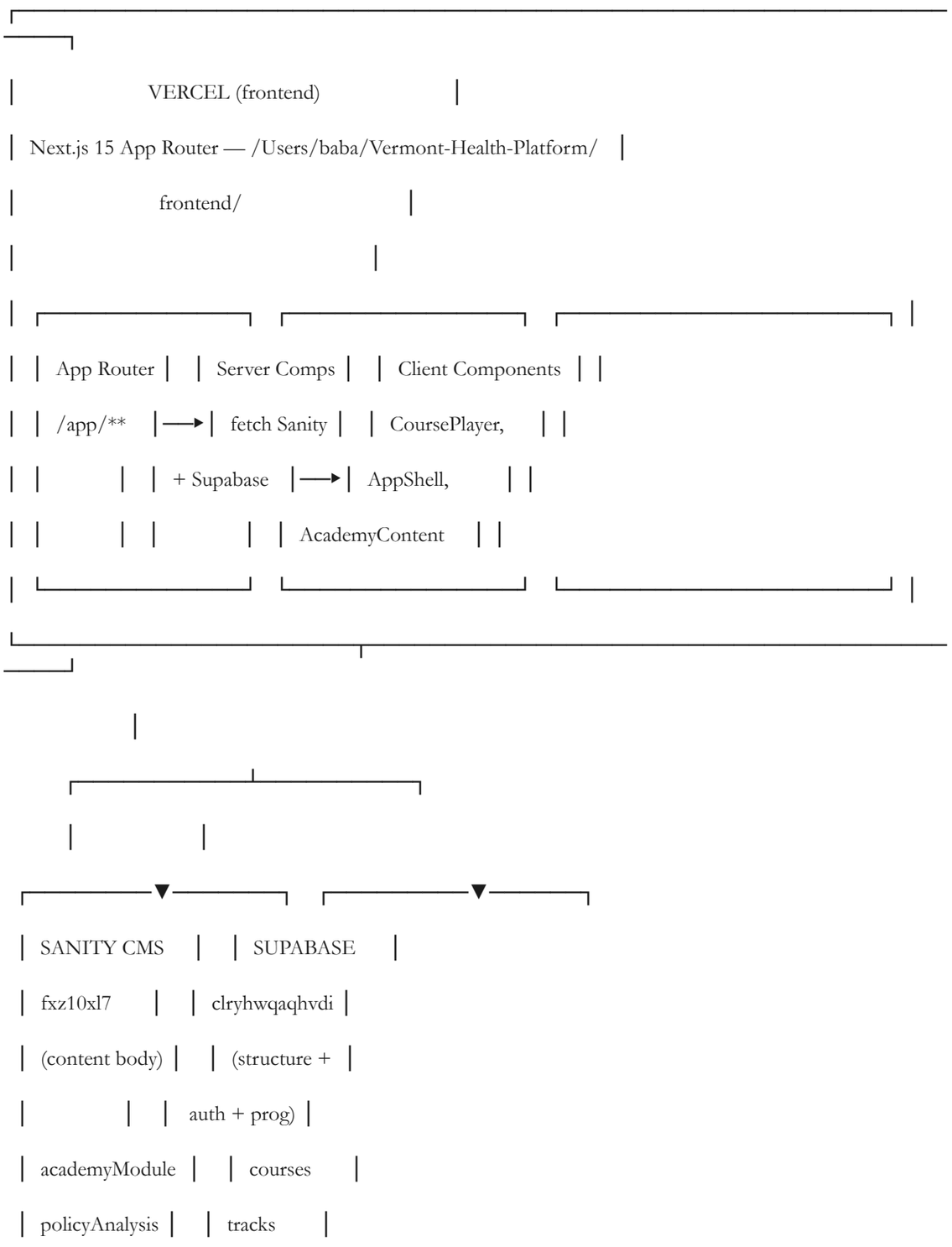
1. Platform Overview

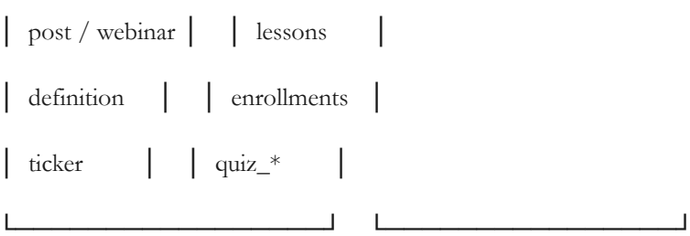
HTR (Healthcare Transformation & Reform) is a knowledge platform for healthcare professionals, policy advocates, and students seeking world-class education on U.S. healthcare transformation. It combines:

- **Academy** — structured courses with lesson players, quizzes, progress tracking, and certifications
- **6-Pillar Content** — policy analysis articles, reports, webinars, and case studies organized by domain
- **AI Analyst** — a GPT-powered analyst accessible via right sidebar and `/chat`
- **Live Data** — ticker, hospital performance, state comparisons, and system vitals dashboards
- **Book Integration** — "Transforming American Healthcare" woven into pillar pages

The platform is production-hosted on **Vercel** (frontend) and **Supabase** (database + auth).

2. Architecture Map





Request flow for a lesson page:

1. Next.js server component loads at `/academy/tracks/[courseSlug]/[lessonSlug]`
2. `getCourseWithProgress()` fetches course + tracks + lessons from Supabase
3. `hydrateWithSanityContent()` batch-fetches all `sanityBody` values from Sanity in one GROQ query
4. Each lesson's `sanityBody` is injected into the lesson object
5. The page renders `CoursePlayer` → `LessonView` → `AcademyContent` (PortableText renderer)

---

3. Content System

3.1 Content Types & Where They Live

Content Type	Primary Store	Secondary
Academy lesson bodies	**Sanity** (`academyModule`)	Supabase `lessons.content_blocks` (legacy fallback)
Course/track/lesson structure	**Supabase** (`courses`, `tracks`, `lessons`)	`frontend/content/courses_tier*.json` (seed source)
Policy analysis articles	**Sanity** (`policyAnalysis`)	—
Blog posts	**Sanity** (`post`)	—
Glossary definitions	**Sanity** (`definition`)	—
Webinars	**Sanity** (`webinar`)	—
Ticker data	**Sanity** (`ticker`)	—
Hospital / state data	**Supabase** (various tables)	`frontend/lib/data/*.ts` (static fallback)
User progress	**Supabase** (`course_lesson_progress`, `course_player_enrollments`)	—

3.2 AI-Generated Content (Claude)

The primary workflow for creating rich Academy lesson content uses Claude Code (this tool) writing directly to the Sanity API via Python + curl. This is the **only reliable method** for large JSON payloads.

Why not the Sanity Studio UI? The Studio is fine for small edits and metadata, but writing 2,000+ word lessons with dozens of rich blocks is impractical by hand. AI generation produces consistent formatting and complete use of all block types.

The pattern used for all course content:

```
import json, subprocess

PROJECT = "fxz10xl7"

DATASET = "production"
```

```
TOKEN   = "<SANITY_API_TOKEN>"          # from frontend/.env.local

API     = f"https://{PROJECT}.api.sanity.io/v2023-10-01/data/mutate/{DATASET}"

# Helper functions that produce valid Sanity block objects

def blk(k,t,s='normal'):

    return {'_type':'block','_key':k,'style':s,'markDefs':[],'children':[{'_type':'span','_key':k+'s','text':t,'marks':[]}]}

def h2(k,t): return blk(k,t,'h2')

def callout(k,t): return blk(k,t,'callout')

def highlight(k,t): return blk(k,t,'highlight')

def stat_grid(k,stats):

    return {'_type':'statGrid','_key':k,'stats':[{'_key':f's{i}','value':s[0],'label':s[1],'context':s[2]} for i,s in
enumerate(stats,1)]}

def warning(k,title,msg): return {'_type':'warningBlock','_key':k,'title':title,'message':msg}

def example(k,title,content): return {'_type':'exampleBlock','_key':k,'title':title,'content':content}

def steps(k,title,items):

    return {'_type':'stepBlock','_key':k,'title':title,'steps':[{'_key':f'step{i}','title':t,'description':d} for i,(t,d) in
enumerate(items,1)]}

def compare(k,title,ll,lp,rl,rp):

    return {'_type':'comparisonBlock','_key':k,'title':title,'left':{'label':ll,'points':lp},'right':{'label':rl,'points':rp}}

def takeaway(k,pts): return {'_type':'takeawayBlock','_key':k,'points':pts}

def quiz(k,q,opts,expl):

    return {'_type':'knowledgeCheck','_key':k,'question':q,'options':[{'_key':f'opt{i}','text':t,'isCorrect':c} for i,(t,c) in
enumerate(opts,1)],'explanation':expl}

def analogy(k,text,concept): return {'_type':'analogyBlock','_key':k,'analogy':text,'concept':concept}

# Post a document

def post(slug, title, body):

    doc = {'_type':'academyModule','_id':slug,'slug':{'_type':'slug','current':slug},'title':title,'body':body}

    payload = {'mutations':[{'createOrReplace': doc}]}

    with open('/tmp/_sp.json','w') as f: json.dump(payload, f)

    r = subprocess.run(['curl','-s','-X','POST',API,

                        '-H','Content-Type: application/json',

                        '-H',f'Authorization: Bearer {TOKEN}',

                        '-d','@/tmp/_sp.json'], capture_output=True, text=True)

    print("OK" if '"error"' not in r.stdout else f"FAIL: {r.stdout[:200]}")
```

Key rules:

- Use `createOrReplace` — safe to re-run, idempotent
- Use the lesson `sanity_slug` value (e.g., `aiml-ethics`) as the document `_id` AND `slug.current`
- Every `_key` in an array must be unique within that document
- Python `True`/`False` (capitalized) — JSON serialization handles the rest
- Never attempt to write large JSON inline in bash heredocs — escaping breaks

3.3 User-Generated Content

Currently, users do not create content directly on the platform. User-generated data is limited to:

- Enrollment records
- Lesson progress (`completed` / `in_progress`)
- Quiz attempts with scores
- Audio uploads for `audio_slot` content blocks (instructor feature)

Future scope: Comment threads, peer Q&A, user-submitted case studies (see Roadmap).

3.4 Platform-Accessible Content

All content served to users flows through the Next.js frontend:

- **Academy lessons** — via `AcademyContent.tsx` rendering Sanity PortableText
- **Pillar pages** — GROQ-fetched from Sanity, rendered as server components
- **Live data dashboards** — queried from Supabase `hospitals`, `state_metrics`, time-series tables
- **AI Analyst** — streamed from the `/api/chat` route via Claude API
- **Glossary** — Sanity `definition` documents, cached 600s

---

4. Academy Course System

4.1 Data Model

courses

└── tracks (ordered groups of lessons within a course)

└── lessons (individual learning units)

├── quizzes

│ └── quiz_questions → quiz_options

└── audio_slots

course_player_enrollments (one per user per course)

└── course_lesson_progress (one per user per lesson)

course_quiz_attempts (one per user per quiz attempt)

The critical link between Supabase structure and Sanity content:

lessons.sanity_slug = "aiml-ethics"

↓

Sanity document: { _type: "academyModule", _id: "aiml-ethics", slug: { current: "aiml-ethics" } }

↓

hydrateWithSanityContent() batch-fetches all slugs → injects as lesson.sanityBody

↓

LessonView checks: lesson.sanityBody → render via AcademyContent

4.2 Adding a New Course

Step 1: Create the JSON seed file

Add the course to `frontend/content/courses_tier1.json` or `courses_tier3.json`. The structure:


```
{
  "id": "unique-uuid-here",
  "slug": "course-slug-here",
  "title": "Course Title",
  "subtitle": "Short subtitle",
  "description": "Full description",
  "targetAudience": ["Clinicians", "Policy professionals"],
  "prerequisites": [],
  "estimatedHours": 10,
  "isPublished": true,
  "version": "1.0.0",
  "createdAt": "2026-01-01T00:00:00Z",
  "updatedAt": "2026-01-01T00:00:00Z",
  "tracks": [
    {
      "id": "unique-uuid-for-track",
      "courseId": "course-uuid",
      "pillar": "technology",
      "order": 1,
      "slug": "track-slug",
      "title": "Track Title",
      "description": "Track description",
      "icon": "ti-brain",
      "targetAudience": [],
      "isPublished": true,
      "createdAt": "2026-01-01T00:00:00Z",
      "updatedAt": "2026-01-01T00:00:00Z",
      "lessons": [
```

```
      {
        "id": "unique-uuid-for-lesson",
        "trackId": "track-uuid",
        "pillar": "technology",
        "order": 1,
        "slug": "lesson-slug",
        "title": "Lesson Title",
        "summary": "1–2 sentence description",
        "estimatedMinutes": 20,
        "objectives": [{"id": "obj1", "text": "Objective text"}],
        "contentBlocks": [],
        "tags": [],
        "isPublished": true,
        "createdAt": "2026-01-01T00:00:00Z",
        "updatedAt": "2026-01-01T00:00:00Z"
      }
    ]
  }
 ]
}
```

Step 2: Seed to Supabase

Navigate to the seed API route or run `seedCourseFromJson()` from `frontend/lib/course-api.ts`:

```
# Via psql for bulk operations:

PGPASSWORD='...' psql "postgresql://postgres.clryhwqaqhvdikgesjbc:...@aws-0-us-west-2.pooler.supabase.com:5432/postgres"

# Or trigger via the admin seed endpoint if wired up
```

The `seedCourseFromJson()` function in `lib/course-api.ts` handles upsert of all tables using `onConflict: "slug"` — safe to re-run.

Step 3: Write lesson content to Sanity (see §4.3)

Step 4: Update `sanity_slug` in Supabase (see §4.4)

4.3 Writing Lesson Content to Sanity

Use the Python + curl pattern from §3.2. Minimum requirements per lesson:

- **2,000+ words** of educational content

- **All rich block types used** at least once: `statGrid`, `exampleBlock`, `comparisonBlock`, `stepBlock`, `knowledgeCheck`, `takeawayBlock`, `warningBlock`, `analogyBlock`, plus `callout` and `highlight` paragraph styles
- **Real statistics** with sources cited in context strings
- **Takeaway block** as the final element summarizing 4–6 key points
- **Knowledge check** (quiz question) in the middle or end of the lesson

Naming convention for `sanity_slug`: `[course-prefix]-[topic-keyword]`

- HIE course: `hie-[topic]` (e.g., `hie-hl7-v2`, `hie-fhir-r4`)
- AI/ML course: `aiml-[topic]` (e.g., `aiml-ethics`, `aiml-governance`)
- Interoperability: `interop-[topic]`
- Revenue cycle: `rcm-[topic]`
- Population health: `pophealth-[topic]`
- Genomics: `genomics-[topic]`

4.4 Wiring Sanity to Supabase

After writing Sanity documents, update `lessons.sanity_slug`:

PGPASSWORD='Bizerte56789!!!TUN' psql "postgresql://postgres.clryhwqaqhvdikgesjbc:Bizerte56789!!!TUN@aws-0-us-west-2.pooler.supabase.com:5432/postgres" << 'SQL'

UPDATE lessons SET sanity_slug = 'your-sanity-slug'

WHERE slug = 'supabase-lesson-slug'

AND track_id IN (

SELECT id FROM tracks WHERE course_id = (

SELECT id FROM courses WHERE slug = 'course-slug-here'

)

);

SQL

Verify with:

SELECT slug, sanity_slug FROM lessons

WHERE track_id IN (SELECT id FROM tracks WHERE course_id = (SELECT id FROM courses WHERE slug = 'your-course'));

4.5 Sanity Rich Block Types Reference

All block types are rendered by `frontend/components/AcademyContent.tsx`.

Block Type	`_type` value	Required fields
Heading (h2/h3)	`block`	`style: "h2"` or `"h3"`, `children[].text`
Body paragraph	`block`	`style: "normal"`, `children[].text`
Callout	`block`	`style: "callout"`, `children[].text`
Highlight	`block`	`style: "highlight"`, `children[].text`
Stat Grid	`statGrid`	`stats: [{value, label, context}]`

Block Type	`_type` value	Required fields
Warning Box	`warningBlock`	`title`, `message`
Example Box	`exampleBlock`	`title`, `content`
Step Block	`stepBlock`	`title`, `steps: [{title, description}]`
Comparison	`comparisonBlock`	`title`, `left: {label, points[]}`, `right: {label, points[]}`
Takeaway	`takeawayBlock`	`points: [string]`
Knowledge Check	`knowledgeCheck`	`question`, `options: [{text, isCorrect}]`, `explanation`
Analogy	`analogyBlock`	`analogy`, `concept`

---

5. Sanity CMS

5.1 Project Details & Access

Item	Value
Project ID	`fxz10xl7`
Dataset	`production`
API Version	`2023-10-01`
Studio URL	`https://htr-platform.vercel.app/studio`
Management	`manage.sanity.io`
Write token	In `frontend/.env.local` as `SANITY_API_TOKEN`

The Studio is embedded in the Next.js app at `/studio` via `frontend/app/studio/[[...index]]/page.tsx`.

5.2 Document Types

`_type`	Purpose
`academyModule`	Academy lesson bodies (the primary content type for the course system)
`policyAnalysis`	Long-form policy articles on the 6 pillar pages
`post`	Blog/news posts
`webinar`	Webinar records with links and dates
`definition`	Glossary terms
`ticker`	Live ticker bar items
`caseStudy`	Case study documents
`report`	Downloadable reports
`analystNote`	Short analyst commentary

_type	Purpose
course	Legacy Sanity course records (superseded by Supabase course system)
instructor	Instructor profiles
hospital	Hospital data (legacy)
rhtState	State-level healthcare data

5.3 blockContent Schema

Defined in `frontend/sanity/schemaTypes/blockContent.ts`. This is the shared rich text schema used for both `policyAnalysis` and `academyModule` bodies. It extends PortableText with custom block types (statGrid, exampleBlock, etc.).

To add a new block type:

1. Add the type definition in `blockContent.ts`
2. Add the renderer in `AcademyContent.tsx`
3. Add the TypeScript type in `frontend/types/course.ts` if used in lessons

5.4 The academyModule Schema

Defined in `frontend/sanity/schemaTypes/academyModule.ts`. Key fields:

- `title` (required string)
- `slug` (required, auto-generated from title)
- `body` (blockContent — the full lesson content)
- `summary` (required text — 3–4 sentences)
- `courseTitle`, `moduleNumber`, `totalModules` (metadata)
- `pillar` (Policy/Economics/Technology/Clinical/Equity/All)
- `level` (Foundational/Intermediate/Advanced)
- `learningObjectives` (array of strings)

Note: When writing via API, only `_type`, `_id`, `slug`, `title`, and `body` are strictly required. Other fields can be filled in the Studio.

5.5 Writing Content via API (Python + curl)

See §3.2 for the complete pattern. Key notes:

- Always use `createOrReplace` — never `create` (will fail if doc exists)
- The `_id` should match `slug.current` for predictable lookups
- File-based curl (`-d @/tmp/_sp.json`) avoids shell escaping issues
- Check response for `"error"` string — successful mutations return `{"transactionId":...,"results":...}`

Verify a document was written:

curl -s "https://fxz10xl7.api.sanity.io/v2023-10-01/data/query/production?query=*[_type=='academyModule' and slug.current=='your-slug']{_id,title}" \

-H "Authorization: Bearer $SANITY_API_TOKEN"

5.6 Bulk Import via bulk_import.js

`scripts/bulk_import.js` reads JSON files from `frontend/sanity/content/` and calls `client.createOrReplace()` for each. Before using:

1. Update `YOUR_PROJECT_ID` to `fxz10xl7`
2. Update `YOUR_WRITE_TOKEN` to the token from `frontend/.env.local`
3. Ensure each JSON file has the correct `_type` and `_id` fields
4. Run: `node scripts/bulk_import.js`

5.7 Cache Invalidation Webhook

Sanity webhooks call `POST /api/webhooks/sanity` when documents are published. The route (`frontend/app/api/webhooks/sanity/route.ts`) calls `revalidateTag()` for the affected document type, busting Next.js's unstable_cache.

Cache TTLs (from `lib/sanity-fetch.ts`):

- `ticker`: 60 seconds
- Feed / lead story: 120 seconds
- `academyModule`: 300 seconds (5 minutes)
- Courses list: 300 seconds
- Glossary: 600 seconds

---

6. Supabase Database

6.1 Connection Details

Item	Value
Project ID	`clryhwqaqhvdikgesjbc`
Direct URL	`postgresql://postgres.clryhwqaqhvdikgesjbc:Bizerte56789!!!TUN@aws-0-us-west-2.pooler.supabase.com:5432/postgres`
Dashboard	`https://supabase.com/dashboard/project/clryhwqaqhvdikgesjbc`
Anon key	In `frontend/.env.local` as `NEXT_PUBLIC_SUPABASE_ANON_KEY`
Service role key	In `backend/.env` as `SUPABASE_SERVICE_ROLE_KEY`

psql shorthand:

PGPASSWORD='Bizerte56789!!!TUN' psql "postgresql://postgres.clryhwqaqhvdikgesjbc:Bizerte56789!!!TUN@aws-0-us-west-2.pooler.supabase.com:5432/postgres"

6.2 Academy Schema Tables

-- Core structure

courses (id, slug, title, subtitle, description, target_audience, prerequisites,
estimated_hours, is_published, version, created_at, updated_at)

tracks (id, course_id, pillar, order, slug, title, description, icon,
target_audience, is_published, created_at, updated_at)

lessons (id, track_id, pillar, order, slug, title, summary, estimated_minutes,
objectives, content_blocks, sanity_slug, ← KEY FIELD
tags, related_lesson_ids, is_published, created_at, updated_at)

quizzes (id, lesson_id, title, passing_score, shuffle_options)

quiz_questions (id, quiz_id, order, question_type, question, explanation, points)

quiz_options (id, question_id, order, text, is_correct, explanation)

audio_slots (id, lesson_id, slot_key, label, hint, uploaded_url, transcript_url,

uploaded_by, uploaded_at)

-- Progress

course_player_enrollments (id, user_id, course_id, status, enrolled_at, completed_at,

percent_complete, current_lesson_id)

course_lesson_progress (id, user_id, lesson_id, enrollment_id, status,

started_at, completed_at)

course_quiz_attempts (id, user_id, quiz_id, lesson_id, answers, score, passed,

completed_at)

The `lessons.sanity_slug` column is the bridge to Sanity. When it is set, `hydrateWithSanityContent()` fetches the lesson body from Sanity and injects it as `lesson.sanityBody`. The `content_blocks` column serves as a fallback when `sanity_slug` is null.

6.3 Common psql Operations

-- List all courses

SELECT id, title, slug FROM courses ORDER BY created_at;

-- List all lessons for a course with Sanity wiring status

SELECT l.slug, l.title, l.sanity_slug

FROM lessons l

JOIN tracks t ON l.track_id = t.id

WHERE t.course_id = 'course-uuid-here'

ORDER BY t.order, l.order;

-- Wire a lesson to Sanity

UPDATE lessons SET sanity_slug = 'sanity-slug-here'

WHERE slug = 'lesson-slug'

AND track_id IN (SELECT id FROM tracks WHERE course_id = 'course-uuid');

-- Check enrollment counts per course

SELECT c.title, COUNT(e.id) as enrollments

FROM courses c

LEFT JOIN course_player_enrollments e ON e.course_id = c.id

GROUP BY c.id, c.title ORDER BY enrollments DESC;

-- Check completion rates

SELECT l.title, COUNT(p.id) as completions

FROM lessons l

LEFT JOIN course_lesson_progress p ON p.lesson_id = l.id AND p.status = 'completed'

GROUP BY l.id, l.title ORDER BY completions DESC LIMIT 20;

6.4 Row-Level Security

RLS is enabled on all user-data tables. The frontend uses two clients:

- `db` (anon key) — for public reads and authenticated user operations within RLS
- `dbAdmin` (service-role key) — bypasses RLS; used only in server-side API routes for seeding and admin operations

Never expose `dbAdmin` to client-side code.

---

7. Authentication

7.1 Supabase Auth Flow

1. User signs up/in via Supabase Auth (email/password or OAuth)
2. Supabase sets a session cookie via `@supabase/ssr`
3. Server components call `createSupabaseServerClient()` from `lib/auth.ts` to read the session
4. The `profiles` table (Supabase public schema) stores role and plan metadata

7.2 User Roles

Defined in `frontend/lib/auth.ts`:

Role	Description
`free`	Default — limited content access
`subscriber`	Paid newsletter/content subscription
`student`	Academy course access
`professional`	Full platform access
`advisory`	Advisory board member
`admin`	Full admin access including Studio

Role hierarchy is enforced via `roleAtLeast(userRole, required)`. Higher index = more access.

7.3 Server-Side Auth Helpers

```
import { getAuthUser, requireAuth, requireRole } from "@/lib/auth";

// In a Server Component:

const user = await getAuthUser();          // returns AuthUser | null

const user = await requireAuth();          // redirects to /login if not authenticated

await requireRole("professional");         // redirects if role insufficient
```

The auth callback route at `frontend/app/auth/callback/route.ts` handles OAuth redirects and sets the Supabase session cookie.

---

8. 6-Pillar Content (Non-Academy)

8.1 The Six Pillars

Pillar	Slug	Description
Policy	`policy`	Healthcare legislation, ACA, Medicaid, Medicare policy

Pillar	Slug	Description
Economics	`economics`	Healthcare costs, financing, value-based care
Technology	`technology`	HIT, EHR, AI, interoperability
Clinical	`clinical`	Care delivery, quality, patient safety
Equity	`equity`	Health disparities, SDOH, access
Operations	`operations`	Hospital operations, workforce, supply chain

Pillar pages live at `/[pillar]` (e.g., `/policy`, `/technology`). They pull `policyAnalysis` documents from Sanity filtered by the `pillar` field.

8.2 Sanity Document Types for Pillars

policyAnalysis — the primary article type

title, summary, pillar, impactLevel (Critical/High/Medium/Low),

publishedAt, body (blockContent), author, tags

post — shorter news/blog posts **webinar** — event records with video links **report** — downloadable PDFs with metadata **caseStudy** — structured case studies

8.3 Adding Pillar Content

Via Sanity Studio (recommended for one-off articles):

1. Go to `https://htr-platform.vercel.app/studio`
2. Click "Policy Analysis" (or relevant type)
3. Fill in title, summary, pillar, impact level, and body
4. Publish → webhook triggers cache revalidation automatically

Via API (for bulk content or AI-generated articles):

```
doc = {
  '_type': 'policyAnalysis',
  '_id': 'policy-article-unique-id',
  'title': 'Article Title',
  'summary': 'Brief summary',
  'pillar': 'policy',  # or technology, economics, clinical, equity, operations
  'impactLevel': 'High',
  'publishedAt': '2026-05-28T00:00:00Z',
  'body': [/* blockContent array */]
}
```

Post using the same Python + curl pattern as Academy lessons.

---

9. Data Ingestion Scripts

9.1 Course Seeding from JSON

`frontend/lib/course-api.ts` exports `seedCourseFromJson(courseData: Course)`. It handles upsert of all tables in order: course → tracks → lessons → audio_slots → quizzes → questions → options.

To trigger it, either:

- Build a small Node/TS script that imports and calls it
- Use the Admin UI seed endpoint (if wired)
- Or manually construct the Supabase upserts via psql

9.2 Python + curl Pattern for Sanity

See §3.2 for the complete reusable pattern. This is the standard for all AI-generated content writes to Sanity.

9.3 bulk_import.js

`scripts/bulk_import.js` — Node.js script for importing pre-authored JSON files. Configure with real credentials before use (placeholder values are in the file). Place JSON documents in `frontend/sanity/content/` directory. Run: `node scripts/bulk_import.js`.

9.4 digest_latest.py

`scripts/digest_latest.py` — Reads the most recent article JSON files from `frontend/sanity/content/` and prints them to stdout for inspection. Useful for verifying content before bulk import.

9.5 Audio Narration Generation

`scripts/generate-narration-audio.sh` and `generate-narration-piper.sh` — Shell scripts for generating audio narration from text using local TTS (Piper). Requires the Piper TTS engine installed in `.piper-venv/`.

---

10. UI Architecture & Maintenance

10.1 Stack

Layer	Technology
Framework	Next.js 15 (App Router)
Language	TypeScript
Styling	Tailwind CSS v4
Components	Custom (no UI component library)
Icons	Lucide React + Heroicons + Tabler Icons
Rich Text	`@portabletext/react`
Auth	`@supabase/ssr`
CMS Client	`next-sanity` + `@sanity/client`

10.2 AppShell & Route Behavior

`frontend/components/AppShell.tsx` wraps every non-studio page. Key route behaviors:

Route pattern	Behavior
`/studio`	No sidebars, no nav
`/chat`	No sidebars (full-screen chat)
`/welcome`	No sidebars
`/academy/tracks/**`	Sidebars hidden, full-height flex layout (course player)
`/academy/tracks` (exact)	Left sidebar forced open on lg+ screens

Route pattern	Behavior
All others	Normal layout with collapsible left/right sidebars

The `isCoursePage` flag (`pathname.startsWith("/academy/tracks/")`) controls the full-height course player layout. Course pages use `h-full overflow-hidden` with `flex-1 min-h-0` to fill the viewport without scrolling the outer shell.

10.3 Course Player Architecture

`frontend/components/course/CoursePlayer.tsx`:

- **Left sidebar:** `CourseSidebar` — fixed 256px, lists tracks and lessons, mobile overlay
- **Content panel:** Scrollable div with drag-adjustable horizontal padding
- **Drag handles:** `position:absolute` handles on the non-scrolling panel; `PAD_MIN=96` (≈1 inch) prevents handles from going off-screen; `PAD_MAX=400`
- **Bottom nav bar:** Prev/Next buttons + progress bar
- **Course complete screen:** Trophy screen with review option

10.4 AcademyContent Renderer

`frontend/components/AcademyContent.tsx` — the gold-standard PortableText renderer. It handles all custom block types. When adding a new block type:

1. Add the Sanity schema block in `blockContent.ts`
2. Add a renderer component in `components/course/content-blocks/` or inline in `AcademyContent.tsx`
3. Register it in the `components` prop of `<PortableText>`
4. Add the TypeScript type in `types/course.ts`

Content priority in `LessonView.tsx`:

1. `lesson.sanityBody` (from `hydrateWithSanityContent`) → render with `AcademyContent`
2. Legacy `sanity_portable_text` block in `lesson.contentBlocks` → render with `AcademyContent`
3. Legacy Supabase `content_blocks` → render with individual block renderers

10.5 Adding a New UI Page

1. Create the route in `frontend/app/[path]/page.tsx`
2. If it's a server component that needs Sanity data, use `cachedFetch` from `lib/sanity-fetch.ts`
3. If it's a full-screen page (like course player), add its path pattern to `AppShell.tsx`
4. If it needs auth, call `requireAuth()` or `requireRole()` at the top of the server component
5. Register any new Sanity document types in `sanity/schemaTypes/index.ts`

---

11. Environment Variables

frontend/.env.local

```
# Supabase

NEXT_PUBLIC_SUPABASE_URL=https://clryhwqaqhvdikgesjbc.supabase.co

NEXT_PUBLIC_SUPABASE_ANON_KEY=<anon-key>

# Sanity

NEXT_PUBLIC_SANITY_PROJECT_ID=fxz10xl7

NEXT_PUBLIC_SANITY_DATASET=production

NEXT_PUBLIC_SANITY_API_VERSION=2023-10-01

SANITY_API_TOKEN=<write-token>            # Server-side only — never expose to client
```

Claude API (for AI Analyst)

ANTHROPIC_API_KEY=<key>

Stripe (subscriptions)

STRIPE_SECRET_KEY=<key>

NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=<key>

STRIPE_WEBHOOK_SECRET=<key>

backend/.env

SUPABASE_DB_URL=postgresql://postgres.clryhwqaqhvdikgesjbc:...@aws-0-us-west-2.pooler.supabase.com:5432/postgres

SUPABASE_URL=https://clryhwqaqhvdikgesjbc.supabase.co

SUPABASE_SERVICE_ROLE_KEY=<service-role-key>

---

12. Deployment & CI/CD

The platform deploys to **Vercel** automatically on every push to `main`.

Build command: `cd frontend && npm run build`
Output directory: `frontend/.next`
Framework preset: Next.js

Build requirements:

- All TypeScript types must be correct — the build will fail on type errors
- `SANITY_API_TOKEN` must be set in Vercel environment variables (it's server-side)
- All `NEXT_PUBLIC_*` variables must be in Vercel project settings

Common build failures:

- Type errors in `frontend/types/course.ts` — any new content block types must be added here
- Missing required fields in Sanity schema used in GROQ queries
- Next.js 15 async APIs (cookies, headers) must be awaited

Checking a deploy:

Check Vercel deploy status

vercel ls --scope=<team>

Or check via GitHub Actions / Vercel dashboard

---

13. Operations Runbook

13.1 Publishing a New Course

1. **Author the course JSON** in `frontend/content/courses_tier*.json` — define course, tracks, and all lesson metadata (title, slug, summary, estimatedMinutes, objectives)
2. **Seed to Supabase** via `seedCourseFromJson()` or direct psql
3. **Write all lesson bodies to Sanity** using the Python + curl pattern
4. **Update `sanity_slug`** in Supabase for every lesson
5. **Verify** by visiting `/academy/tracks/[course-slug]` and clicking through lessons
6. **Set `is_published = true`** on the course (or ensure it's set in the JSON before seeding)

13.2 Editing an Existing Lesson

Minor text edits → use Sanity Studio at `/studio` — find the `academyModule` document by slug, edit, publish. Cache expires within 5 minutes or on next webhook.

Major content rewrites → use the Python + curl `createOrReplace` pattern. Provide the same `_id` (slug) to overwrite the existing document.

Structural changes (add a lesson, reorder lessons) → update `lessons.order` in Supabase via psql. Reordering does not require Sanity changes.

Quiz changes → update `quiz_questions` and `quiz_options` tables in Supabase via psql or the admin UI.

13.3 Adding a New Pillar Article

1. Go to Sanity Studio → Policy Analysis (or relevant type)
2. Create new document with title, summary, `pillar`, `impactLevel`, `publishedAt`
3. Write body content using Studio rich text editor
4. Publish — webhook auto-invalidates the cache for that pillar

For AI-generated articles at scale, use the Sanity API pattern with `_type: 'policyAnalysis'`.

13.4 Monitoring & Alerts

Vercel: Check deploy logs at vercel.com/dashboard. Build failures send email to the deployment account.

Supabase: Check project health at supabase.com/dashboard. Monitor DB size, API usage, and auth events.

Sanity: Content publish activity at manage.sanity.io. API usage dashboard shows query rates.

Application errors: No custom error tracking currently wired (Sentry not yet installed). Server-side errors appear in Vercel function logs.

Key metrics to monitor manually:

- Enrollment count per course (query `course_player_enrollments`)
- Lesson completion rates (query `course_lesson_progress`)
- Quiz pass rates (query `course_quiz_attempts`)
- Sanity cache hit rate (Vercel analytics → cache headers)

---

14. Content Status — All Courses

Course	Lessons	Sanity Bodies	Supabase Wired	Status
HIE & Health Reform	13	13 ✓	13 ✓	**Complete**
AI & Machine Learning	18	18 ✓	18 ✓	**Complete**
Healthcare Interoperability	25	0	0	Not started
Revenue Cycle Management	21	0	0	Not started
Population Health Management	17	~1	~1	Partial
Genomics & Precision Medicine	21	6	6	Partial (15 remaining)
Medicaid 101	~10	?	?	Legacy
Medicaid Managed Care	~10	?	?	Legacy

Course	Lessons	Sanity Bodies	Supabase Wired	Status
Value-Based Care	~10	?	?	Legacy

Next priority order: Healthcare Interoperability → Revenue Cycle Management → Population Health Management → Genomics & Precision Medicine.

---

15. Improvement & Scaling Roadmap

Near-Term (Next 3 Months)

Content completions:

- Write all 25 Healthcare Interoperability lessons to Sanity
- Write all 21 Revenue Cycle Management lessons
- Complete Population Health Management (16 remaining)
- Complete Genomics (15 remaining)

Platform quality:

- Wire Sentry for error tracking (`@sentry/nextjs`)
- Add lesson-level analytics events (started, completed, time-on-lesson)
- Add quiz attempt analytics (questions answered, time taken, score distribution)
- Build admin dashboard for content editors to monitor course health

Content authoring tools:

- Build a simple web form for non-technical content authors to draft lesson metadata (title, summary, objectives) before AI writes the body
- Create a lesson preview mode that shows Sanity content in the course player without requiring deployment

Medium-Term (3–12 Months)

User experience:

- Certificates of completion with verifiable hashes (infrastructure exists in `certifications` table)
- Personalized learning paths based on role, pillar interest, and completion history
- Search across all lessons and pillar content (Supabase full-text or Algolia)
- Mobile-optimized course player (current design is desktop-first)
- Note-taking panel alongside lesson content

Content expansion:

- Learner-facing Q&A on each lesson (Supabase discussion threads)
- Expert interviews woven into lessons as embedded audio/video
- Interactive case studies with branching scenarios
- Vermont-specific case studies embedded throughout relevant lessons

Technical:

- Edge caching for Sanity GROQ responses (currently 5-min server cache)
- Streaming lesson content for faster time-to-first-content
- ISR (Incremental Static Regeneration) for high-traffic lesson pages
- Database read replicas for analytics queries without affecting production

Long-Term (12+ Months)

Platform scale:

- Multi-tenant support (white-label for health systems, health plans, academic programs)
- LMS integrations (SCORM/xAPI export for enterprise customers)
- Continuing education credit (CME/CE) certification pipeline
- Cohort-based learning with instructor-led tracks

AI integration depth:

- AI-powered study companion per lesson (context-aware Q&A using lesson body as context)
- Adaptive quizzes that adjust difficulty based on learner performance
- AI-generated lesson summaries and flashcards
- Automated lesson quality scoring and improvement suggestions

Infrastructure:

- Move from Vercel hobby to Vercel Pro/Enterprise for higher function limits
- Supabase Pro for larger DB, PITR backups, and custom domains
- Consider Sanity Growth plan for higher API request limits as content volume grows
- CDN for audio assets (current audio slots have no CDN)

Last updated: 2026-05-28. Maintained by: bechir.bensaid@gmail.com